MPCache: Model-Predictive Caching for Training-Free Video Diffusion Acceleration

Anonymous Authors[1]

Abstract

Video diffusion transformers are costly to sample: every step runs the denoiser twice under classifier-free guidance. Training-free caching replaces some evaluations with cached features or cheap forecasts, but chooses when to skip and what to return with separate rules, so a cheap-looking skip can still return a value whose error later steps amplify. Inspired by model-predictive control, we propose MPCache, which identifies one small model of the denoising dynamics online, at zero extra denoiser calls, and uses it three ways: State-Conditioned Forecasting predicts skipped steps from the current latent, Solver-Aware Pricing propagates the forecast error to the terminal step, and a Residual Check updates the model and its trust region at every exact evaluation. A receding-horizon planner then allocates the evaluation budget across the trajectory. A skip now carries an explicit cost before it is taken rather than being discovered after. Weights and sampling grids are untouched. Experiments on CogVideoX-2B and Wan2.1-T2V-1.3B demonstrate improved efficiency–fidelity trade-offs over TeaCache, TaylorSeer, and RA-CFGCache at matched exact-evaluation budgets. Code is available at `https://github.com/Yizhou-Yang/MPCache`.

1. Introduction

Diffusion transformers have become important backbones for image and video generation (Peebles & Xie, 2023; Yang et al., 2025; Team Wan et al., 2025). Their inference cost is dominated by repeated denoiser evaluations over a sampling trajectory, with long video token sequences making each evaluation expensive. Training-free caching offers an attractive alternative to retraining: retain the pretrained network and sampling schedule, but replace some expensive computations with previously cached quantities or inexpensive predictions. Two decisions govern the resulting quality–efficiency trade-off: *what should a skipped computation return, and when is that approximation worth its downstream cost?*

Existing methods address complementary parts of this problem. TeaCache uses changes in timestep-modulated inputs as inexpensive proxies for output variation and controls reuse with a rescaled, accumulated signal (Liu et al., 2025a). TaylorSeer forecasts intermediate features from finite-difference estimates of temporal derivatives (Liu et al., 2025b). RA-CFGCache aligns cache decisions with classifier-free guidance (CFG) and rescales local guided-risk estimates using an offline-calibrated, timestep-dependent propagation prior (Liu et al., 2026). These approaches are not simply equivalent to treating the denoiser as a time series: TeaCache already observes input-dependent signals, and RA-CFGCache explicitly accounts for downstream propagation. The remaining question is whether a *causal, per-trajectory dynamics model* can improve both the approximation itself and the choice of where to spend exact computation.

The distinction matters because forecasting and scheduling interact through the latent state. Write a guided denoiser as $v_i = f(x_i, \tau_i)$, where x_i is the current latent and τ_i the scheduler's noise coordinate. A temporal forecaster estimates changes along the trajectory represented by its cached observations. Once a prediction is used, however, the sampler evolves to a different latent; the next prediction is queried on this altered trajectory. Similarly, two forecast errors of equal magnitude can have different downstream effects because the solver and denoising field transform their directions differently. A scheduling rule cannot generally evaluate one forecast independently of the states and cache anchors induced by earlier choices. This motivates joint selection, not a claim that local proxies are always inferior.

MPCache addresses this coupling by caching a local model rather than only an output. At exact evaluation points, it records latent–output pairs and fits a channel-diagonal

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

plus low-rank affine approximation to the guided field. At skipped steps, the current latent is already available from the sampler and can be used directly in that approximation. The fitted operator also defines a local tangent model of the sampling update, yielding a surrogate for the propagation of forecast error. The controller uses that surrogate to compare compute, reuse, temporal extrapolation, and state-conditioned prediction under a remaining evaluation budget. When a new exact evaluation arrives, its pre-update prediction residual tests the model, tightens or relaxes the trust region, and triggers replanning. Thus forecasting, pricing, and feedback refer to the same identified object rather than three independently calibrated components.

This is a discrete-action instance of model predictive control: identify a model from available observations, predict the consequences of candidate actions, execute one action, and repeat (Rawlings et al., 2017). The analogy is operational, not a transfer of classical MPC stability guarantees. A denoising trajectory provides limited excitation; its time drift and state response can be confounded. A model that predicts outputs along that trajectory may still misrepresent the off-trajectory directions needed for error propagation. Our initial channelwise finite-difference diagnostic did not establish useful propagation pricing. We therefore treat propagation fidelity as an empirical gate, not as an already demonstrated consequence of caching a local model. The proposed structured, causal estimator and the full controller remain to be validated.

The intended methodological contributions are:

- **One model for two coupled decisions.** A regularized local guided-field model is shared by state-conditioned forecasting and solver-aware cost estimation, with explicit limits on what trajectory observations can identify.
- **Budgeted receding-horizon cache control.** A controller jointly selects refresh times and prediction operators while retaining scheduler and cache-history state; approximate search is distinguished from exact dynamic programming.
- **Residual-based model checking.** Exact evaluations provide causal, pre-update diagnostics that govern trust and replanning without additional probe evaluations.

These are design contributions in the present draft, not claims of verified performance. The evaluation target is a common protocol on CogVideoX-2B and Wan2.1-T2V-1.3B against TeaCache, TaylorSeer, and RA-CFGCache, with matched budgets, paired fidelity measurements, and separately measured runtime overhead. A successful result must demonstrate that the shared model improves the joint decision, not merely that any one of its components is useful.

2. Related Work

Training-free reuse and forecasting. TeaCache estimates nonuniform output variation from inexpensive, timestep-modulated inputs and rescales the estimated changes for adaptive caching (Liu et al., 2025a). Its input sensitivity is important: our distinction is not simply that prior work ignores the current latent. TaylorSeer replaces direct reuse with Taylor-series feature forecasts whose derivatives are approximated from historical feature values (Liu et al., 2025b). MPCache instead proposes an explicit state-and-time model of the guided output. Its temporal fallback is an output-level first-order extrapolator, not a reproduction of TaylorSeer's intermediate-feature implementation. Comparisons must use the actual baseline code and identify any model-specific adaptation.

Guidance- and propagation-aware cache control. RA-CFGCache identifies both a branch-guided mismatch and a local-final mismatch under CFG (Liu et al., 2026). It composes branch-wise error proxies using guidance coefficients and calibrated cross-branch alignment, then applies a timestep-dependent propagation prior before online thresholding. Consequently, propagation-aware caching itself is prior work, not our novelty claim. The proposed distinction is a causal, prompt-specific local operator shared by output prediction and finite-horizon action selection. Whether that operator adds information beyond a timestep prior is unresolved and must be measured; substituting such a prior into MPCache is a necessary ablation, not evidence for online identification.

Numerical sampling. DDIM and UniPC reduce the number of denoising evaluations through sampling schemes with different update rules and solver histories (Song et al., 2021; Zhao et al., 2023). MPCache keeps the selected sampling grid and scheduler fixed and changes which steps receive exact network outputs. Its propagation model must therefore use the actual scheduler, including prediction-type conversion and multistep history. A one-step tangent formula is not automatically valid for a predictor–corrector scheduler.

Online identification and predictive control. Secant and quasi-Newton methods construct derivative approximations from function evaluations without forming a dense Jacobian (Broyden, 1965). We borrow this limited-memory identification principle, but fitting along a denoising trajectory is not equivalent to observing fixed-time state perturbations. Model predictive control repeatedly optimizes a finite-horizon action sequence and executes only its first ac-

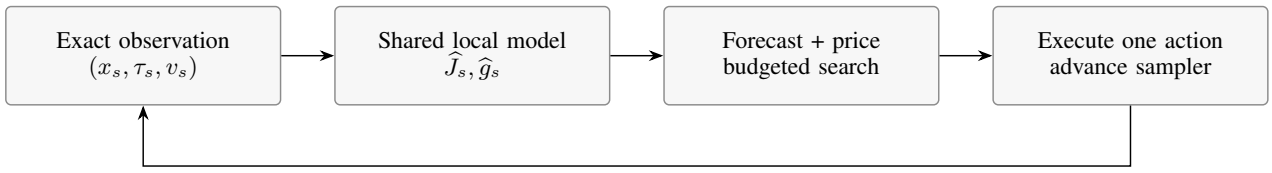


Figure 1. MPCache design. A causal local model is shared by forecasting and propagation-aware planning; the feedback observation is an ordinary budgeted evaluation, not an additional probe.

tion (Rawlings et al., 2017). MPCache uses that structure for compute allocation; low-rank fitting, receding-horizon planning, and residual checks are not individually claimed as new. The research hypothesis is that coupling them through one useful dynamics model yields a better cache controller.

Video models and evaluation. CogVideoX and Wan provide the two video backbones used in the planned comparison (Yang et al., 2025; Team Wan et al., 2025). VBench supplies a multidimensional video-evaluation framework (Huang et al., 2024). Using its prompt resource for paired LPIPS, SSIM, and PSNR is distinct from computing a full VBench score. We follow RA-CFGCache's shared generation protocol for reproducibility and will report these evaluation scopes separately.

3. Method: Model-Predictive Caching

3.1. Problem Setup and Causal Information

Fix the text conditioning and guidance scale for a sample. Let $i = 0, \dots, T-1$ index the original denoising grid in execution order, with latent $x_i \in \mathbb{R}^d$ and normalized noise coordinate τ_i. The coordinate is the scheduler's noise level, not necessarily the integer step index. We model the solver-facing guided output

$$v_i = f(x_i, \tau_i) = u_i + \omega(c_i - u_i), \tag{1}$$

where u_i and c_i are unconditional and conditional predictions in the same parameterization. Prediction-type conversions are part of the scheduler adapter. An exact action evaluates both branches; our first design does not select branch-wise refreshes. One exact guided evaluation costs one budget unit, with its two branch evaluations or batched equivalent reported separately.

Write the scheduler as

$$z_{i+1} = S_i(z_i, \widetilde{v}_i), \qquad x_i = P z_i, \tag{2}$$

where z_i includes the latent and any solver history. For a deterministic affine one-step sampler, this reduces to $x_{i+1} = a_i x_i + b_i \widetilde{v}_i$. Multistep and predictor–corrector sampling require the augmented form, not that simplification. Caching changes $\widetilde{v}_i$, not the grid or solver.

At step i, only previous exact evaluations and the current sampler state are available. The anchor buffer is $\mathcal{B}_i = \{(x_s, \tau_s, v_s) : s < i \text{ was exact}\}$. The current exact output, future reference latents, and suffix reruns are unavailable to the controller. A total budget B includes warmup and all forced refreshes. Full-trajectory terminal fidelity is an evaluation target; the controller optimizes an estimated surrogate, since the all-exact terminal output is not known online.

3.2. A Structured Local Guided-Field Model

Let s be the most recent exact anchor. Our local affine model is

$$m_s(x, \tau) = v_s + \widehat{J}_s(x - x_s) + \widehat{g}_s(\tau - \tau_s). \tag{3}$$

With C latent channels, let $D(\theta)$ multiply each channel by its scalar θ_c. Let $Q_s \in \mathbb{R}^{d \times r}$ have orthonormal columns spanning up to r recent latent differences. We restrict

$$\widehat{J}_s = D(\theta_s) + R_s Q_s^\top, \qquad R_s \in \mathbb{R}^{d \times r}. \tag{4}$$

This gives a channel-diagonal response outside the observed subspace and a low-rank correction inside it; no dense $d \times d$ matrix is formed.

For consecutive exact anchors in a buffer of at most $m+1$ observations, construct columns $\Delta x_j = x_{s_j} - x_{s_{j-1}}$, $\Delta v_j = v_{s_j} - v_{s_{j-1}}$, and $\Delta \tau_j = \tau_{s_j} - \tau_{s_{j-1}}$. Collect them in X, Y, and row vector δ. Fit

$$\min_{\theta, R, g} \left\| (Y - D(\theta)X - RQ_s^\top X - g\delta) W^{1/2} \right\|_F^2 + \lambda_D \|\theta\|_2^2 + \lambda_R \|R\|_F^2 + \lambda_g \|g - g_0\|_2^2, \tag{5}$$

where W is a positive diagonal recency weighting and all ridge coefficients are positive. The temporal prior g_0 is the slope between the two most recent exact outputs divided by their noise-coordinate separation; without two distinct anchors, $g_0 = 0$ and skipped actions are disabled. The model shrinks toward temporal extrapolation, rather than interpreting every output difference as a state derivative. Gram matrices and the small coupled solves use FP32; field storage

can use the inference dtype. The rank, buffer size, and ridge settings must be fixed on development prompts before evaluation.

Innovation form. The temporal coefficient can be removed exactly rather than regularized. Require the model to interpolate the two most recent exact anchors $s' < s$. With $r_i = (\tau_i - \tau_s)/(\tau_s - \tau_{s'})$, Equation (3) then becomes

$$m_s(x_i, \tau_i) = \underbrace{v_s + r_i(v_s - v_{s'})}_{\text{temporal extrapolation}} + \widehat{J}_s d_i, \qquad d_i = (x_i - x_s) - r_i(x_s - x_{s'}). \tag{6}$$

We call d_i the *state innovation*. The first term is the first-order temporal forecast. The second corrects it only by the part of the current latent that the same straight-line extrapolation in τ fails to predict. Every earlier exact anchor k with two predecessors $k'' < k' < k$ supplies one training pair without an extra evaluation: the innovation $d_k = (x_k - x_{k'}) - r_k(x_{k'} - x_{k''})$ and the temporal-forecast residual $e_k = v_k - [v_{k'} + r_k(v_{k'} - v_{k''})]$. To first order, $e_k \approx J d_k$. The ridge problem of Equation (5) is solved on these pairs. Shrinking $\widehat{J}_s$ toward zero therefore recovers temporal extrapolation, the regularization target stated above. With the channel-diagonal part only, the fit has a closed form: per channel, divide the recency-weighted sum of $d_k e_k$ by $(1 + \lambda_D)$ times the weighted sum of d_k^2.

Identification limit. Raw anchor differences satisfy $\Delta v \approx J\Delta x + g\Delta\tau$, not the fixed-time secant equation $\Delta v \approx J\Delta x$, and time and latent motion are strongly correlated along one trajectory. The innovation form removes the component of state motion that is linear in τ. The fitted operator is thus driven by deviations from straight-line motion, which are much less collinear with time; curvature of the temporal drift remains confounded with Jd. Regularization still chooses a model rather than making the physical Jacobian identifiable. In particular, $\widehat{J}_s$ is a fitted response operator on innovation directions, and its behavior outside them is uncertain. A successful output forecast is not evidence that this operator correctly propagates arbitrary perturbations. This distinction is central to the validation gate.

3.3. State-Conditioned Forecasting

We allow four actions: exact computation C, reuse R, temporal extrapolation T, and model prediction M. For the three skipped actions,

$$\begin{aligned} \widetilde{v}_i^{\mathsf{R}} &= v_s, \\ \widetilde{v}_i^{\mathsf{T}} &= v_s + g_0(\tau_i - \tau_s), \\ \widetilde{v}_i^{\mathsf{M}} &= \widetilde{v}_i^{\mathsf{T}} + \gamma_i\big[m_s(x_i, \tau_i) - \widetilde{v}_i^{\mathsf{T}}\big]. \end{aligned} \tag{7}$$

Here T is an output-level fallback, not TaylorSeer's feature-cache implementation; with g_0 from the two latest anchors it coincides with the first term of Equation (6). By Equation (6), $m_s - \widetilde{v}_i^{\mathsf{T}} = \widehat{J}_s d_i$. The coefficient $\gamma_i = \beta_s \min\{1, \kappa \left\|\widetilde{v}_i^{\mathsf{T}} - v_s\right\| / \left\|\widehat{J}_s d_i\right\|\}$ has two parts. The trust factor caps the correction at κ times the temporal increment. The self-check gate $\beta_s \in (0, 1]$ is updated at every exact evaluation by comparing the forecast the old model would have made there with temporal extrapolation: β_s increases additively when the model wins and is halved when it loses.

Define normalized latent displacement $d_x = \|x_i - x_s\| / (\|x_s\| + \varepsilon)$, time displacement $d_\tau = |\tau_i - \tau_s|$, and unseen displacement $d_\perp = \left\|(I - Q_s Q_s^\top)(x_i - x_s)\right\| / (\|x_s\| + \varepsilon)$. A trust gate accepts model prediction only when these are below current radii and the anchor age is below a maximum gap. The clipping rule and gates are applied to actual forecasts, not only to planning costs. Reuse and temporal actions also have age and residual gates; they are not automatically safe fallbacks.

Let q_s be a robust scale of recent *pre-update* absolute model residuals, with an explicit positive floor during warmup. A concrete residual envelope is

$$q_i = q_s(1 + \kappa_x d_x + \kappa_\tau d_\tau + \kappa_\perp d_\perp + \kappa_a(i - s)), \tag{8}$$

with nonnegative coefficients fixed on development prompts. We price a skipped action by

$$\widehat{\ell}_i^a = \|\widetilde{v}_i^a - m_s(x_i, \tau_i)\| + q_i, \qquad \widehat{\ell}_i^{\mathsf{C}} = 0. \tag{9}$$

The envelope is a heuristic uncertainty model, not a calibrated coverage guarantee. Including q_i is essential: without it, the shared model could incorrectly assign zero error to its own prediction.

3.4. Solver-Aware Pricing

For one isolated output error $e_i = \widetilde{v}_i - f(x_i, \tau_i)$ followed by exact evaluations, first-order perturbations of an affine one-step solver obey

$$\delta x_{i+1} = b_i e_i, \qquad \delta x_{j+1} = (a_j I + b_j J_j)\delta x_j. \tag{10}$$

Consequently, the estimated terminal impulse is

$$\widehat{\delta x}_T = \left[\prod_{j=T-1}^{i+1} (a_j I + b_j \widehat{J}_j)\right] b_i e_i, \tag{11}$$

where the product is ordered in execution order from right to left. This illustrates why a local norm alone can miss downstream amplification. At decision time, all future $\widehat{J}_j$ must be generated from the current fitted model and its trust envelope; they are not estimated using future exact outputs.

The initial implementation holds $\widehat{J}_s$ fixed within a planning rollout and shortens the horizon when this assumption leaves the trust region.

For the actual scheduler, use its augmented transition. Let $A_j = \partial_z S_j$ and $B_j = \partial_v S_j$ in solver-facing coordinates. The all-exact local tangent is $A_j + B_j \widehat{J}_j P$. For a candidate policy, the state must additionally include cache anchors and forecast history; differentiation uses the executed forecast rule, including clipping, and cache updates. Reuse and temporal forecasts have zero current-latent derivative only when their anchors are held fixed. Refreshing a cache removes a new local approximation error but *does not reset accumulated latent error*. A scheduler adapter that omits these histories cannot support a claim of closed-loop propagation fidelity.

In reduced planning coordinates, propagate an uncertainty radius r_j through the policy tangent:

$$r_{j+1} = \widehat{\alpha}_j^a r_j + \widehat{\beta}_j^a \widehat{\ell}_j^a, \tag{12}$$

where $\widehat{\alpha}_j^a$ and $\widehat{\beta}_j^a$ are amplification estimates from the shared operator, solver coefficients, and cache-history transition. They can be obtained by a fixed number of matrix-free norm iterations, with an explicit off-subspace margin. The latent dimension and the small history are scaled consistently in this norm. The radius is initialized to zero at warmup, carried across subsequent replans, and never reset merely because the next action is exact. Neither approximate norm estimation nor a fitted operator makes r_j a certified bound. A short-horizon rollout uses r_{i+H} plus a solver-only tail multiplier as its terminal risk surrogate; a full remaining-horizon rollout requires no tail approximation. Latent risk is also not equated with perceptual video quality.

3.5. Budgeted Receding-Horizon Planning

Let y_i contain the sampler state, anchor buffer, fitted model, trust variables, and propagated risk estimate. For remaining budget b_i, plan

$$\begin{aligned} &\min_{\pi=(a_i,\dots,a_{i+H-1})} \widehat{\mathcal{R}}(\pi; y_i) \\ &\text{s.t.} \quad \sum_{j=i}^{i+H-1} \mathbf{1}[a_j = \mathsf{C}] \le b_i - b_{\text{tail}}. \end{aligned} \tag{13}$$

subject to trust and maximum-gap constraints. Here b_{tail} reserves mandatory refreshes outside the horizon. The risk $\widehat{\mathcal{R}}$ is the terminal radius surrogate from Equation (12), including the tail multiplier when required. Runtime and model-fitting overhead are measured separately; this first controller targets exact-evaluation count, not guaranteed wall-clock optimality.

A prospective exact action uses m_s as its nominal mean output, costs one budget unit, and updates the *simulated* anchor history. It introduces zero new caching error but does not provide an observed residual. Planning never calls the expensive network and never treats a simulated anchor as real identification data. The fitted operator and observed uncertainty scale stay frozen during the rollout; trust distances are measured against the last *real* identification anchor even after simulated refreshes. Only execution yields a new observation and permits refitting.

For these frozen surrogate dynamics, an exact Bellman recursion would have state (i, b, y):

$$\begin{aligned} V_i(b, y) = \min_{a \in \mathcal{A}(y)} V_{i+1}(b', y'), \\ b' = b - \mathbf{1}[a = \mathsf{C}], \quad y' = F_i^a(y). \end{aligned} \tag{14}$$

with terminal value $V_{i+H} = \widehat{\mathcal{R}}$ and infeasible states assigned $+\infty$. The continuous sampler and cache history make this large; a table indexed only by step and budget is not exact dynamic programming. Our proposed practical approximation is beam search retaining at most K distinct states per depth and budget, ranked by current risk plus a common tail heuristic. It preserves candidate-dependent nominal states and anchor choices rather than merging them by budget alone. Only the first action of the best feasible sequence is executed. Search is repeated from the actual sampler state at the next step, and the model is re-identified only after exact evaluation.

Warmup uses at least two exact anchors and is charged to B. Reserve refreshes according to the maximum-gap constraint, including a final exact step when configured. If no action remains feasible within budget, report budget–trust infeasibility rather than silently exceeding B or claiming a safe prediction. A separately labeled quality-priority mode may spend extra evaluations, but those evaluations must be counted and are excluded from hard matched-budget claims.

3.6. Residual Checks and Replanning

Before an exact evaluation at s' is incorporated, compute the old model's residual

$$\epsilon_{s'} = \frac{\|f(x_{s'}, \tau_{s'}) - m_s(x_{s'}, \tau_{s'})\|}{\|f(x_{s'}, \tau_{s'})\| + \varepsilon}. \tag{15}$$

Store the corresponding absolute residual for q_s. If the residual exceeds its predicted envelope, shrink the trust radii, reduce the maximum permitted gap, and increase uncertainty. After a fixed number of consecutive in-envelope observations, relax these variables conservatively within preset limits. Then append the new exact pair, refit Equation (5), invalidate the previous plan, and replan using the remaining budget. The residual is evaluated before refitting, so it cannot be made artificially small by fitting the

Algorithm 1 MPCache (proposed causal controller)

Require: Pretrained guided field f, scheduler S, budget B
Initialize real anchor buffer, risk, trust, and budget
for $i = 0$ to $T - 1$ **do**
  if warmup or mandatory refresh **then**
    Choose $a_i = \mathsf{C}$ if budget-feasible
  else
    Build candidate forecasts from real observations
    Search feasible model rollouts; take first action
  end if
  if no feasible action **then**
    Report budget–trust infeasibility; stop
  end if
  if $a_i = \mathsf{C}$ **then**
    Evaluate both branches; form v_i by Eq. (1)
    Check old model before updating; adjust trust
    Append exact anchor, refit model, decrement budget
    Set $\widetilde{v}_i = v_i$; discard old plan
  else
    Set $\widetilde{v}_i$ to the chosen gated forecast
  end if
  Propagate risk without resetting prior latent error
  Advance actual scheduler state $z_{i+1} = S_i(z_i, \widetilde{v}_i)$
end for

observation being checked. It measures the model on the realized cached trajectory, not the unknown deviation from an all-exact reference.

3.7. Cost, Scope, and Validation Requirements

Applying Equation (4) costs $O(dr)$ operations and $O(dr)$ model storage; the real anchor buffer costs $O(dm)$. Constructing a rank-r basis from m differences and fitting the small coupled system also incurs reduction and factorization overhead. A beam with K states per budget, horizon H, and four actions scales at least as $O(HKb_i|\mathcal{A}|)$ surrogate transitions; each transition has its own low-rank and scheduler cost. Full-dimensional beam states can be memory intensive, so practical compression requires a further fidelity check. Zero additional *network evaluations for fitting* does not mean zero latency or memory overhead. No backbone training or custom CUDA kernel is required by this design.

The current method is a specification, not a completed end-to-end implementation. A previous diagnostic using fixed-time, channelwise finite differences required extra network calls and used future trajectory information when estimating suffix propagation. It therefore does not validate this causal, zero-probe controller, and its failed pricing gate cannot be repaired by baseline engineering fixes. Required next tests are: (i) causal propagation estimates versus a leave-one-prompt-out timestep prior, including within-timestep gain variation; (ii) state forecasts versus reuse and temporal extrapolation under closed-loop sampling; and (iii) the full controller versus the same forecasts with fixed or prior-based schedules at matched exact-evaluation budgets. The propagation component must improve on the prior before the shared-model hypothesis is claimed as supported. Paired LPIPS, SSIM, and PSNR, separately scoped VBench evaluation, runtime, peak memory, and failure rates are all needed for the eventual empirical claim.

4. Experiments

Models, prompts, and fidelity. We adopt the protocol of RA-CFGCache Table 1 (Liu et al., 2026) so that our numbers can be placed next to the three closest training-free baselines without re-tuning them. CogVideoX-2B (Yang et al., 2025) generates 49 frames at 720×480 and 8 fps with 50 sampling steps and guidance scale 6. Wan2.1-T2V-1.3B (Team Wan et al., 2025) generates 81 frames at 832×480 and 16 fps with 50 UniPC steps (Zhao et al., 2023), flow shift 3, and guidance scale 5. Prompts are the non-empty lines of the VBench prompt resource (Huang et al., 2024) used by that harness, of which there are 100; prompt k uses seed k, negative prompt "animation", and batch size one. Every accelerated video is compared with the full 50-step video of the same prompt and seed by LPIPS (AlexNet), SSIM, and PSNR. Using the VBench prompt list does not constitute a VBench evaluation; reference-free VBench dimensions are reported separately when they are run.

Baselines. TeaCache (Liu et al., 2025a) uses thresholds 0.2 (CogVideoX) and 0.15 (Wan); TaylorSeer (Liu et al., 2025b) uses interval 3, order 1, and three warmup steps; RA-CFGCache-Slow/Fast use thresholds 0.6/1.4 (CogVideoX) and 0.4/0.6 (Wan) with the released calibration tables. All baselines run inside the pinned public RA-CFGCache harness, which also contains the TeaCache and TaylorSeer implementations, with two corrections that do not change any method. First, the pinned sampling script forwarded the command-line threshold under a key that the RA-CFGCache controller does not read, so both nominal variants silently ran at the default threshold; we bind the threshold explicitly and assert the effective value at run time. Second, all denoising and caching run on the GPU without CPU offload.

Hardware deviations. The published numbers use an H200 in bf16. We run the identical protocol on four H20-96GB cards in bf16, with no CPU offload, no VAE tiling, and no text-encoder parking, so the precision matches the paper and the only deviation is the accelerator itself. A single V100-32GB in fp16 was used as a cross-check: it

reproduces CogVideoX and the four Wan rows at n=32, but its 31.7 GiB cannot hold TaylorSeer's per-layer derivative cache on Wan, which needs roughly 90 GiB across the two CFG branches. Latencies are chunk averages measured while four cards are busy, so they are provisional; the quality columns are the comparable ones. All wall-clock numbers for the controller, including its online-identification overhead, will be re-timed on an otherwise idle device.

Reproduced baselines. Table 1 reports all 100 prompts of both backbones; every row has 100 paired samples and no row is imputed. Across the five rows that the published table also contains, LPIPS differs by at most 0.017, SSIM by at most 0.015 and PSNR by at most 0.5 dB, and the ordering is identical: RA-CFGCache-Slow is the strongest fidelity baseline on both models, TeaCache and RA-CFGCache-Fast trade places with the backbone, and 25 steps is the weakest non-trivial row. This makes RA-CFGCache-Slow the primary comparison for any matched-budget claim. The speedups are not comparable to the published ones: they are chunk averages on four busy H20 cards, and H20 is not H200. TaylorSeer on Wan is reproduced here for the first time in this project (0.3496 LPIPS at $2.47\times$); it is clearly worse than TeaCache at a comparable budget, the same pattern as the published Wan row, so its poor CogVideoX result is a property of the method's first-order time extrapolation and not an implementation error. One prompt (id 56 on CogVideoX) collapses to a white clip for the reference and for every method; excluding it moves the row means by at most 0.004 LPIPS and changes no ordering. No statistical gate is claimed for the baselines.

Development platform and component gates. Components of MPCache are first tested on a cheaper development setting: CogVideoX-2B, 17 frames at 720×480, 30 DDIM steps (Song et al., 2021), guidance 6, and eight fixed prompts with two seeds each (16 paired samples). These prompts are disjoint from the evaluation prompts. Each component must pass a pre-registered gate before it enters the full controller. For the forecaster, all methods compute exactly the same steps, given by the static TeaCache schedule at $N \in \{10, 13, 17\}$ exact evaluations, and differ only in what skipped steps return. Fidelity is the relative ℓ_2 error of the terminal latent against the all-exact run. The forecaster passes if no sample diverges and the paired median error decreases by at least 10% relative to the better of reuse and first-order temporal extrapolation at every budget. For propagation pricing, a causal estimate must rank the terminal effect of single-step perturbations better than a leave-one-prompt-out timestep prior of the RA-CFGCache type. The full controller is then compared at matched exact-evaluation budgets with the same forecasts under fixed and prior-based schedules, with paired tests requiring at least 16 samples, a median improvement of at least 25%, a win rate of at least 0.75, and an effect larger than twice its standard error.

Limitations. MPCache has no validated result. Both pre-registered component gates fail: the online propagation price ranks terminal error at Spearman 0.33 against 0.95 for a timestep prior, and the state-conditioned forecaster beats the better of reuse and first-order extrapolation by 9.1% at N=13 but loses 22% at N=10. The controller has therefore never been run end-to-end at the scale of Table 1, which reports baselines only. The reproduction also runs on four H20 cards rather than the published H200, so only the quality columns are comparable; one CogVideoX prompt collapses to a white clip for the reference and for every method; and each baseline is evaluated at the budget its own scheduler selects, so its wall-clock advantage over a fixed budget is not uniform.

References

Broyden, C. G. A class of methods for solving nonlinear simultaneous equations. *Mathematics of Computation*, 19(92):577–593, 1965. doi: 10.1090/S0025-5718-1965-0198670-6. URL https://www.math.unm.edu/~vageli/courses/Ma576/papers/Broyden_65.pdf.

Huang, Z., He, Y., Yu, J., Zhang, F., Si, C., Jiang, Y., Zhang, Y., Wu, T., Jin, Q., Chanpaisit, N., Wang, Y., Chen, X., Wang, L., Lin, D., Qiao, Y., and Liu, Z. VBench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 21807–21818, 2024. URL https://arxiv.org/abs/2311.17982.

Liu, F., Zhang, S., Wang, X., Wei, Y., Qiu, H., Zhao, Y., Zhang, Y., Ye, Q., and Wan, F. Timestep embedding tells: It's time to cache for video diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7353–7363, 2025a. URL https://arxiv.org/abs/2411.19108.

Liu, J., Zou, C., Lyu, Y., Chen, J., and Zhang, L. From reusing to forecasting: Accelerating diffusion models with TaylorSeers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025b. URL https://arxiv.org/abs/2503.06923.

Liu, Y., Wan, B., Liu, T., Wang, A., Xiong, Y., Zhang, F., Chen, H., and Ding, G. RA-CFGCache: From branch-level criteria to guided-risk control under classifier-free guidance. arXiv preprint arXiv:2609.36433, 2026. URL https://arxiv.org/abs/2609.36433. NeurIPS 2026.

Table 1. Reproduction of the RA-CFGCache Table 1 protocol with the pinned public harness (first n VBench prompts, seed = prompt index, fidelity vs. the same-seed 50-step output). "paper" is the published H200/bf16 value; ours uses four H20-96GB in bf16 under the identical protocol, with no CPU offload and no VAE tiling. †Speedup is a chunk-averaged ratio; H20 and H200 are different silicon, so only its trend is comparable. Every row has 100 paired samples; non-numeric entries would mark incomplete rows and are never filled with zeros. TaylorSeer on CogVideoX has no published reference value in that table.

Model	Method	n	LPIPS↓ ours / paper	SSIM↑ ours / paper	PSNR↑ ours / paper	Speedup† ours / paper
CogVideoX-2B	Full (T=50)	100	0.0000 / 0.0000	1.0000 / 1.0000	∞ / ∞	1.00× / 1.00×
	Fewer steps (T=25)	100	0.4704 / 0.4751	0.5350 / 0.5445	14.08 / 13.61	1.89× / 1.86×
	TeaCache	100	0.2122 / 0.2040	0.7392 / 0.7538	21.22 / 21.48	2.64× / 2.42×
	TaylorSeer (N=3, O=1)	100	0.4047 / –	0.5626 / –	15.38 / –	2.36× / –
	RA-CFGCache-Slow	100	0.0990 / 0.0968	0.8661 / 0.8746	26.89 / 26.93	2.63× / 2.39×
	RA-CFGCache-Fast	100	0.2092 / 0.2052	0.7493 / 0.7614	21.88 / 21.89	3.74× / 3.13×
Wan2.1-T2V-1.3B	Full (T=50)	100	0.0000 / 0.0000	1.0000 / 1.0000	∞ / ∞	1.00× / 1.00×
	Fewer steps (T=25)	100	0.4204 / 0.4371	0.5700 / 0.5658	15.85 / 15.87	1.96× / 1.93×
	TeaCache	100	0.1681 / 0.1661	0.7887 / 0.7955	22.84 / 23.08	2.53× / 2.44×
	TaylorSeer (N=3, O=1)	100	0.3496 / 0.3632	0.6283 / 0.6187	17.10 / 16.96	2.47× / 2.13×
	RA-CFGCache-Slow	100	0.0941 / 0.0993	0.8820 / 0.8778	28.11 / 28.03	2.65× / 2.56×
	RA-CFGCache-Fast	100	0.1352 / 0.1409	0.8420 / 0.8386	26.14 / 26.12	3.14× / 2.99×

Peebles, W. and Xie, S. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023. URL https://arxiv.org/abs/2212.09748.

Rawlings, J. B., Mayne, D. Q., and Diehl, M. M. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2 edition, 2017. URL https://sites.engineering.ucsb.edu/~jbraw/mpc/MPC-book-2nd-edition-1st-printing.pdf.

Song, J., Meng, C., and Ermon, S. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021. URL https://arxiv.org/abs/2010.02502.

Team Wan, Wang, A., Ai, B., Wen, B., Mao, C., Xie, C.-W., Chen, D., Yu, F., Zhao, H., Yang, J., Zeng, J., et al. Wan: Open and advanced large-scale video generative models. arXiv preprint arXiv:2503.20314, 2025. URL https://arxiv.org/abs/2503.20314.

Yang, Z., Teng, J., Zheng, W., Ding, M., Huang, S., Xu, J., Yang, Y., Hong, W., Zhang, X., Feng, G., Yin, D., Zhang, Y., Wang, W., Cheng, Y., Xu, B., Gu, X., Dong, Y., and Tang, J. CogVideoX: Text-to-video diffusion models with an expert transformer. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2408.06072.

Zhao, W., Bai, L., Rao, Y., Zhou, J., and Lu, J. UniPC: A unified predictor-corrector framework for fast sampling of diffusion models. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2302.04867.